TP 19 : La compétition du lundi

Ce TP est à réaliser dans le langage que vous souhaitez.

Deuxième TP sous cette forme, vous êtes rassemblés en équipe pour réaliser ce TP. L'objectif est simple : obtenir le plus de réponses. Pour cela, chaque équipe devra compléter une feuille qu'elle me rendra à la fin du TP. Chaque équipe aura un u_0 qui lui est propre que je vous fournis en début d'heure. Pour savoir si vos programmes sont corrects, vous pouvez comparer vos résultats avec ceux obtenus pour $u_0 = 10$ à la fin chaque partie.

Préliminaires

Si $a > 0$ et $b > 0$ sont deux entiers, on note $a \mod b$ le reste de la division euclidienne de a par b, autrement dit l'unique entier r avec $0 \leq r < b$ tel qu'il existe un entier q satisfaisant $a = bq + r$.

Chaque partie commence par une génération de nombre pseudo-aléatoires. Cette sous-partie est nécessaire pour la suite. Vous pouvez réaliser les questions dans l'ordre que vous le souhaitez. Vous pouvez travailler à plusieurs ou vous répartir les questions. Une fois que vous avez la réponse à une question, vous pouvez venir écrire votre réponse au tableau. Vous pouvez modifier vos réponses jusqu'à la fin de l'heure où l'on décomptera les réponses justes.

Attention, dans ce sujet, on manipule de très grands entiers, potentiellement trop grand pour être simplement stocké sur 32 bits. Pour ne pas avoir d'erreurs évitables, vérifiez que vous manipulez des entiers encodés sur 64 bits. Vous rencontrerez généralement cette situation en travaillant sur TryOCaml. En C, utilisez des entiers de type `long` pour éviter ce problème. Si jamais vous voyez es nombres négatifs sortir des modulos, c'en est probablement la cause.

1 Plus proche voisin

Dans cet exercice on s'intéresse à rechercher dans un ensemble de vecteurs réels X de dimension d l'élément le plus proche, au sens du carré de la distance euclidienne, d'un vecteur requête $y \in Y$. Ce problème, dit de la "recherche du plus proche voisin", est un classique de l'informatique qui apparaît dans un grand nombre de domaines, notamment en apprentissage machine.

Génération de suites aléatoires

Étant donné u_0, on définit la suite suivante :

$$\forall k \in \mathbb{N}, u_{k+1} = (19\ 999\ 999 \times u_k) \mod 19\ 999\ 981$$

Question 1 : Calculer les valeurs suivantes :

- $u_1 \mod 1000$
- $u_{98} \mod 1000$
- $u_{9\ 876\ 543} \mod 1000$

À partir de trois entiers positifs d, n, et m, on en déduit n vecteurs $(x_i)_{0 \leq i < n}$ de dimension d, et m vecteurs $(y_j)_{0 \leq j < m}$ de dimension d :

$$\forall 0 \leq k < d \begin{cases} x_i[k] = u_{id+k} \mod 1000 \\ y_j[k] = u_{(n+j)d+k} \mod 1000 \end{cases}$$

où $x_i[k]$ (respectivement $y_j[k]$) désigne la k-ième coordonnée de x_i (respectivement y_j). On notera dorénavant $X = \{x_0, \ldots, x_{n-1}\}$ et $Y = \{y_0, \ldots, y_{m-1}\}$. Par ailleurs, on notera z un vecteur quelconque dans la suite de l'énoncé.

Question 2 : Calculer les valeurs de $x_{33}[13]$ et $y_{123}[43]$ pour les valeurs suivantes :

- $n = 1000$, $m = 10000$ et $d = 128$
- $n = 1000$, $m = 10000$ et $d = 200$
- $n = 100$, $m = 10000$ et $d = 128$

Recherche naïve

On appelle plus proche voisin de y_j dans X le vecteur x défini par :

$$x = \arg\min_{x_i \in X} (||x_i - y_j||_2)^2$$

, c'est-à-dire le vecteur de X le plus proche au sens du carré de la distance Euclidienne du vecteur y_j. Notons qu'il est possible que plusieurs x_i soient simultanément les plus proches voisins du vecteur y_j . On choisira alors systématiquement celui d'indice le plus faible dans le reste de cet énoncé. Cette notion est étendue à celle de k-ième plus proche voisin, en retirant de X un à un $k-1$ fois le plus proche voisin, puis en calculant le plus proche voisin sur l'ensemble restant.

Enfin, on ne cherchera pas à vérifier si tous les vecteurs x_i sont distincts deux à deux. Dans untel cas, seul celui d'indice le plus faible pourra être considéré un plus proche voisin d'un vecteur donné.

Pour trouver les plus proches voisins d'un vecteur y_j , on propose dans un premier temps de réaliser une recherche exhaustive. Elle consiste à calculer pour tout $0 \leq i < n$ les distances $(||x_i - y_j||_2)^2$ puis à identifier ensuite les plus proches voisins. On qualifie cet algorithme de naïf. On répète ce procédé indépendamment pour tous les vecteurs de Y.

Question 3 : Pour $n = 100\ 000$, $m = 1$ et les valeurs de d suivantes, déterminer l'indice du plus proche voisin dans X de y_0 :

- $d = 1$
- $d = 32$
- $d = 128$

On appelle représentativité de x_i dans X et pour Y le nombre de fois que x_i est le plus proche voisin d'un vecteur de Y (on rappelle pour la dernière fois que si un vecteur y_j est à distance minimale de plusieurs vecteurs dans X, seul celui d'indice le plus faible est considéré).

Question 4 : Pour $n = 100$, $m = 10\ 000$ et $d = 32$, calculer les représentativités de :

- x_{12}
- x_{34}
- x_{56}

Soit z un vecteur de $[\![0, 999]\!]^d$. On note $r(z)$ la représentativité de z dans $X_z = \{x_0, , x_{n-1}, z\}$ et pour Y. En d'autres termes, on s'intéresse à la représentativité de z lorsqu'il est ajouté en tant que dernier élément (d'indice n) dans X

Question 5 : Pour $d = 1$, déterminer $\max_{z \in [\![0,999]\!]} r(z)$ pour les paramètres suivants :

- $n = 3$ et $m = 1000$
- $n = 3$ et $m = 10\ 000$
- $n = 3$ et $m = 100\ 000$

Valeurs pour $u_0 = 10$

Question	a	b	c
1	180	913	918
2	(966, 29)	(544, 190)	(966, 467)
3	3064	97704	83391
4	94	144	318
5	758	7620	75784

TABLE 1 – Résultats des différentes questions

2 Compression de données

Génération de nombres pseudo-aléatoires

Étant donné u_0, on définit la suite suivante :

$$\forall k \in \mathbb{N}, u_{k+1} = (28 \times u_k) \mod 336\ 529$$

Question 1 : Calculer les valeurs suivantes :
- $u_1 \mod 1000$
- $u_{42} \mod 1000$
- $u_{10^5} \mod 1000$

Génération de sources aléatoires

Un mot w de taille $n = |w|$ sur un alphabet Σ est constitué de n lettres $w_0 \dots w_{n-1}$ de Σ.

En théorie de l'information, on définit une source comme un objet qui produit de façon aléatoire une séquence de symboles, ce qui nous intéresse étant la probabilité d'apparition de chaque symbole. Dans ce sujet, pour simplifier on suppose qu'une source est un mot dont les symboles sont des lettres de Σ. Ainsi la probabilité d'apparition d'un symbole de S est son nombre d'occurrences dans S, divisé par la longueur de S.

L'entropie d'une source S dont chaque symbole x_i apparaît avec probabilité $p_i > 0$ est donnée par

$$H(S) = -\sum_i p_i \log_2(p_i)$$

et s'exprime en bits. L'entropie est le nombre minimal de bits pour encoder un symbole en moyenne.

Compresser est une opération qui consiste à coder, c'est-à-dire transformer une source S en un code C qui peut être représenté par un certain nombre de bits. Dans ce sujet on s'intéresse à une compression sans perte, ce qui veut dire qu'il est toujours possible de décoder, c'est-à-dire récupérer la source originale à partir du code.

Soit b un entier supérieur ou égal à 2, K un entier positif. Un réel $r \in [0,1]$ peut s'écrire comme un mot infini $a_1 a_2 \dots$ de symboles de $\{0, \dots, b-1\}$ tel que

$$r = \sum_{i=1}^{+\infty} a_i b^{-i}$$

Notez que cette écriture n'est pas nécessairement unique, comme par exemple $0,999\dots = 1$ en base 10. À des fins de terminaison, on s'intéressera à une écriture tronquée à des mots de K caractères et l'on acceptera toutes les écritures possibles.

L'alphabet comporte $M+1$ lettres de 0 à M. Dans ce sujet, $M = 29$. La source S_N est le mot de taille $N+1$ dont le t-ième symbole $S_N[t]$ pour $t = 0, \dots, N$ vaut :

$$\begin{cases} S_N[t] = u_t \mod M & \text{si } t < N \\ S_N[N] = M \end{cases}$$

Ainsi, la source S_N termine toujours par le symbole de fin de séquence M.

Question 2 : Calculer les 5 derniers symboles de S_N pour les valeurs de N suivantes :
- $N = 10$
- $N = 1000$
- $N = 100\ 000$

Question 3 : Calculer l'entropie de la source S_N pour les valeurs de N suivantes, arrondie à 5 chiffres après la virgule.
- $N = 10$
- $N = 1000$
- $N = 100\ 000$

Codage de Huffman

On s'intéresse à une première méthode de compression qui s'appuie sur la construction d'un arbre binaire, dont l'arête vers le fils gauche (resp. droit) est étiquetée par 0 (resp. 1). Les feuilles représentent les symboles de la source, tandis que le chemin de la racine à une feuille est le mot binaire pour coder le symbole correspondant à cette feuille. Le code de la source est la concaténation des codes de ses symboles. Idéalement, on souhaite que les lettres fréquentes de la source soient codées avec des mots binaires courts.

L'algorithme de construction de l'arbre de Huffman s'appuie sur une structure de données S qui contient des arbres associés à des scores, et procède comme suit :

Algorithme de codage de Huffman

1. Pour chaque symbole de la source (c'est-à-dire, apparaissant au moins une fois), insérer dans S une feuille étiquetée par le symbole, ayant pour score son nombre d'occurrences.
2. Tant qu'il y a plus d'un arbre dans S
   (a) Extraire de S les deux arbres A_1 , A_2 ayant les plus faibles scores p_1 et p_2 . En cas d'égalité des scores, l'arbre ajouté le plus tôt dans S sort de S en premier.
   (b) Créer un nouvel arbre ayant pour sous-arbre gauche A_1 et sous-arbre droit A_2.
   (c) Ajouter ce nouvel arbre à S avec le score $p_1 + p_2$.
3. L'unique arbre restant dans S est l'arbre de Huffman.

Question 4 : Calculer la hauteur de l'arbre de Huffman associé à la source S_N pour les valeurs de N suivantes :
- $N = 10$
- $N = 1000$
- $N = 100\ 000$

Question 5 : Calculer la taille de message de la source S_N pour les valeurs de N suivantes :
- $N = 10$
- $N = 1000$
- $N = 100\ 000$

Codage arithmétique

On s'intéresse à une nouvelle méthode de compression appelée codage arithmétique, inspirée par Shannon et aujourd'hui utilisée dans l'encodage vidéo H.264. L'idée consiste à diviser l'intervalle dans lequel on se trouve en zones proportionnellement à la fréquence de chaque symbole. Les zones sont ordonnées par l'ordre naturel sur les symboles. On part de l'intervalle $[0,1[$ avec $L_0 = 0$ et $D_0 = 1$. Pour chaque symbole de la source, on emprunte la zone correspondant à ce symbole et on met à jour l'intervalle $[L_i, L_i + D_i[$. La source finit par le symbole de fin de séquence M qui permet d'identifier à quel moment interrompre le codage et surtout le décodage. Le code de la source peut enfin être n'importe quel nombre qui appartient à l'intervalle $[L_n, L_n + D_n[$ où n est la longueur de la source. Il est conseillé de faire un exemple pour bien comprendre (ou de me poser des questions).

Question 6 : Pour cette question on attend pour chaque valeur de N un nombre compris entre 0 et 1 arrondi à N chiffres en base 10 après la virgule. Implémenter le codage arithmétique et donner la valeur de L_{N+1} obtenue en compressant la source S_N pour :
- $N = 7$
- $N = 10$
- $N = 15$

Question 7 : Décoder les cinq premières lettres du code $\frac{u_0}{N}$ en supposant savoir que la fréquence de la source correspond à S_N pour les valeurs de N suivantes :
- $N = 100$
- $N = 500$
- $N = 1000$

Valeurs pour $u_0 = 10$

Question	a	b	c
1	280	929	313
2	$[14,4,25,10,29]$	$[20,28,9,8,29]$	$[27,13,8,0,29]$
3	2.84535093662	4.84330209998	4.8578457997
4	5	7	7
5	32	4889	492610
6	0.299101829529	0.373348486835	0.463895887363
7	$[4,13,25,4,7]$	$[0,22,20,0,26]$	$[0,12,0,15,13]$

TABLE 2 – Résultats des différentes questions

3 Suites universelles

Génération de nombre aléatoires

On pose $m = 2^{31} - 1 = 2\ 147\ 483\ 647$, $a = 46\ 613$ et $b = 17$. Étant donné u_0, on définit la suite suivante :

$$\forall k \in \mathbb{N}, u_{k+1} = (a \times u_k + b) \mod m$$

Question 1 : Calculer les valeurs suivantes :
- $u_1 \mod 1000$
- $u_{98} \mod 1000$
- $u_{9876} \mod 1000$

Manipulation de suites

On considère des suites d'entiers naturels non-nuls. Pour une suite $l = (a_0, a_1, \ldots, a_{k-1})$, on note $|l|$ sa longueur, ici égale à k. Les préfixes de l sont les suites de la forme $(a_0, a_1, \ldots, a_{p-1})$ pour $p \leq k$ et les suffixes de l sont les suites de la forme $(a_p, a_{p+1}, \ldots, a_{k-1})$ pour $p \geq 0$.

Étant donnés n, k, α, on définit la suite $B(n,k,\alpha) = (a_0, \ldots, a_{k-1})$ où $a_i = (u_{\alpha i} \mod n) + 1$.

La capacité d'une suite est la somme de ses éléments.

Question 2 : Calculer la capacité des suites $B(n,k,\alpha)$ suivantes :
- $B(9,5,13)$
- $B(98,54,17)$
- $B(987,543,41)$
- $B(98\ 765, 54\ 321, 97)$

On dit qu'une suite $l_1 = (a_0, \ldots a_{k-1})$ domine une autre suite $l_2 = (b_0, \ldots b_{p-1})$ s'il existe

$$0 \leq i_0 < \ldots < i_{p-1} \leq k-1$$

tels que pour tout $j \in [\![0, p-1]\!]$, on a $b_j \leq a_{i_j}$.

Par exemple (5, 3, 1, 10) domine (4, 9) mais (5, 3, 1, 10) ne domine pas (6, 4).

Question 3 : Calculer pour un entier b et une liste $B(n,k,\alpha)$ le premier indice i tel que $b \leq a_i$, et -1 s'il n'en existe pas dans les cas suivants :
- $b = 9,\ n = 9,\ k = 5,\ \alpha = 13$
- $b = 86,\ n = 98,\ k = 54,\ \alpha = 17$
- $b = 975,\ n = 987,\ k = 543,\ \alpha = 41$
- $b = 98123,\ n = 98765,\ k = 54321,\ \alpha = 97$

Question 4 : Calculer pour une suite l_1 et une suite l_2 la longueur du plus grand préfixe l_2' de l_2 tel que l_1 domine l_2' pour les suites suivantes :
- $l_1 = B(9,5,13)$ et $l_2 = B(9,10,14)$
- $l_1 = B(98,54,17)$ et $l_2 = B(98,68,43)$
- $l_1 = B(987,543,41)$ et $l_2 = B(987,673,174)$
- $l_1 = B(98\ 765, 54\ 321, 97)$ et $l_2 = B(98\ 765, 65\ 431, 81)$

Une suite l est n-universelle si elle domine toute suite de capacité n. Par exemple, la suite (1, 3, 1) est 3-universelle. En effet, voici toutes les suites de capacité 3 :

$$(1,1,1)\ (1,2)\ (2,1)\ (3)$$

Nous étudions la construction suivante : $L_0 = ()$, $L_1 = (1)$ et L_n est la suite obtenue en concaténant $L_{\lfloor n/2 \rfloor}$, puis (n), puis $L_{n-1-\lfloor n/2 \rfloor}$. Par exemple $L_2 = (1,2)$, $L_3 = (1,3,1)$, $L_4 = (1,2,4,1)$, et $L_5 = (1,2,5,1,2)$.

Question 5 : Calculer la capacité de L_n pour les valeurs de n suivantes :
- 6
- 987
- 98 765
- 9 876 543

Énumérer les suites de capacité bornées

Fixons n, on définit un ordre total sur les suites de capacité au plus n : $l_1 = (a_0, \ldots, a_{k-1}) <_1 l_2 = (b_0, \ldots, b_{p-1})$ si :

- soit l_1 est un préfixe de l_2,
- soit l_1 n'est pas un préfixe de l_2 et il existe $i < k$ tel que pour tout $j < i$, $a_j = b_j$ et $a_i < b_i$.

Par exemple, voici l'énumération des suites de capacité au plus 3 dans l'ordre $<_1$:

$$() \ (1) \ (1,1) \ (1,1,1) \ (1,2) \ (2) \ (2,1) \ (3)$$

Question 6 : Calculer pour une suite l et un entier n, l'indice de l dans la suite (indexé depuis 0) énumérant les suites de capacité au plus n dans l'ordre $<_1$, et -1 si l a capacité plus que n

- $l = B(2,4,13) \quad n = 7$
- $l = B(3,6,17) \quad n = 15$
- $l = B(3,8,41) \quad n = 20$
- $l = B(5,8,97) \quad n = 25$

On définit un deuxième ordre total sur les suites de capacité au plus n : $l_1 = (a_0, \ldots, a_{k-1}) <_2 l_2 = (b_0, \ldots, b_{p-1})$ si :

- soit $k < p$,
- soit $k \geq p$ et il existe $i < k$ tel que pour tout $j < i$, $a_j = b_j$ et $a_i < b_i$.

Par exemple, voici l'énumération des suites de capacité au plus 3 dans l'ordre $<_2$:

$$() \ (1) \ (2) \ (3) \ (1,1) \ (1,2) \ (2,1) \ (1,1,1)$$

Question 7 : Calculer pour une suite l et un entier n, l'indice de l dans la suite (indexé depuis 0) énumérant les suites de capacité au plus n dans l'ordre $<_2$, et -1 si l a capacité plus que n

- $l = B(2,4,13) \quad n = 7$
- $l = B(3,6,17) \quad n = 15$
- $l = B(3,8,41) \quad n = 20$
- $l = B(5,8,97) \quad n = 25$

Résultats pour $u_0 = 10$

Question	a	b	c	d
1	147	96	882	
2	19	2919	269276	2684228332
3	-1	1	21	30
4	3	15	156	682
5	14	?	?	?
6	20	21766	748912	16084852
7	68	7589	212665	1006005

TABLE 3 – Résultats des différentes questions